

Lab 3 – Architectural Justification

Faculty Research Grant & Publication Tracker | Nivas Reddy Dandu | PES1UG24AM075 | Section B

Architecture Selection

We chose a Layered Architecture for the Faculty Research Grant & Publication Tracker System.

The system is organised into three layers: a Presentation Layer (Web Portal UI for Faculty Researchers and the Research Dean), a Business Layer (Grant & Expense Manager, Publication Manager, Approval Workflow, Analytics Service) and a Data Layer (Database Repository and an immutable Audit Ledger). Each layer talks only to the layer directly below it through well-defined interfaces (IGrantService, IPublicationService, IApprovalReview, IAnalyticsService, IApprovalRequest, IDataAccess, IAuditLog).

Reasons for the Choice

- 1. Fits the scale and nature of the system.** The tracker is a departmental, transaction-oriented application with a single shared dataset (grants, expenses, publications, approvals) and a modest, predictable user base. Microservices would add network latency, distributed-data consistency problems and operational overhead with no real benefit, while a plain client-server design would mix UI and business rules on the server. Layering gives clear separation of concerns at the right scale.
- 2. Business rules are centralised and consistently enforced.** The most critical rule in the system – an expense must never exceed the grant cap (FR-002) – lives in exactly one place, the Grant & Expense Manager in the Business Layer. Every request, whether from a researcher's form or the Dean's analytics view, passes through this layer, so the rule cannot be bypassed by the UI, and the same balance logic feeds the fund burn-up analytics (FR-005), keeping figures consistent.

Security Advantage

Strict layering means the Presentation Layer can never access the database directly; all data operations go through authenticated REST/HTTPS business interfaces and then through the IDataAccess interface using parameterised SQL, which narrows the attack surface and prevents SQL injection. In addition, the Database Repository writes every change to financial approvals and publication status through the IAuditLog interface into an append-only, hash-chained Audit Ledger. Because no upper layer has a path to modify the ledger, audit history cannot be silently altered or deleted – directly satisfying NFR-001 (tamper-evident audit trail).

Performance Benefit

Because each layer is independent, the Business and Data Layers can be tuned without touching the UI. Frequently read data such as grant balances, burn-up totals and publication metrics can be cached in the Business Layer and served by the Analytics Service, and the stateless business layer can be replicated behind a load balancer during peak periods (e.g., grant reporting deadlines). In-process calls between business components (such as Publication Manager → Approval Workflow) avoid the network hops a microservices design would need, keeping response times low under expected peak load (NFR-002).